Autonomous Agent for Malware Detection

Advanced Topics in Cyber Defense

Ofek Bar-Shalom
Ariel University
barshalo.ofek@msmail.ariel.ac.il

April 15, 2026

Abstract

We present an autonomous agent built with LangGraph and powered by a local LLM via Ollama that continuously collects, analyzes, and learns from PE (Portable Executable) malware samples. The system features an LLM-guided data acquisition pipeline that intelligently discovers and retrieves samples from MalwareBazaar and VirusTotal, extracts and validates over 60 static PE features, trains an XGBoost classifier, and detects concept drift to trigger model retraining. The LLM guides search strategy selection, interprets drift detection results, and generates iteration summaries. Our approach demonstrates the feasibility of building a self-improving malware detection system that adapts to evolving threats without manual intervention.

1 Introduction

Malware evolves rapidly, and signature-based detection struggles to keep pace, motivating ML-based approaches that nonetheless face continuous data collection needs, concept drift, and repeated retraining. In this work we develop an autonomous LangGraph agent that addresses these challenges through a cyclic pipeline of data collection, feature extraction, model training, evaluation, and drift detection, operating continuously to discover new malware sources, collect PE samples, and improve its classifier over time.

2 Related Work

2.1 Static PE Feature Analysis

Anderson and Roth [1] introduced EMBER, a 2,351-feature PE dataset (headers, sections, imports, byte histograms) with a LightGBM baseline, establishing static PE analysis at scale. EMBER is reproducible and comprehensive but fixed and drift-unaware. Our extractor borrows EMBER's feature categories (headers, imports, sections, entropy) in a lighter 60-feature form optimized for continuous extraction, and feeds it from a live collection loop rather than a frozen dataset.

2.2 Concept Drift in Malware Detection

Jordaney et al. [2] (Transcend) showed malware classifiers can decay to near-random within months and proposed conformal evaluation for drift detection — statistically grounded but detection-only, calibration-hungry, and Android-focused. This motivated our PSI+DDM drift layer, which we couple to automatic retraining rather than stopping at detection.

Pendlebury et al. [3] (TESSERACT) exposed temporal bias in malware ML evaluation and gave a principled temporal-split framework, though it is evaluation-only with no mitigation. Because our agent collects chronologically, train/test splits are naturally temporally ordered, which aligns with TESSERACT's recommendations.

2.3 Autonomous Security Agents

Chen et al. [4] surveyed LLM agents for software engineering and security ops, cataloguing tool-augmented architectures; the survey is broad but light on concrete systems and mostly assumes cloud LLMs (GPT-4), raising cost and privacy concerns. We adopt the tool-augmented pattern but run locally via Ollama, removing the cloud dependency.

2.4 Continual Learning

Al-Essa and Kalita [5] (FL-MalDrift) couple on-device drift detection with adaptive federated aggregation — privacy-preserving but complex, coordination-heavy, and per-client detection can miss global shifts. We keep the drift-triggered-update idea but collapse it to a single-agent, centrally-monitored setting.

Xu et al. [6] (MADCAT) adapt at test time via self-supervision, needing no labels but risking noise and catastrophic forgetting. We take the complementary route: detect drift, then fully retrain on freshly collected labeled samples, trading latency for a cleaner adaptation.

2.5 Summary and Positioning

Table 1: Comparison of related approaches.

Approach	**Autonomous**	**Drift**	**Continual**	**Agent**	**Local LLM**
EMBER [1]	No	No	No	No	No
Transcend [2]	No	Yes	No	No	No
TESSERACT [3]	No	Yes	No	No	No
FL-MalDrift [5]	Partial	Yes	Yes	No	No
MADCAT [6]	Partial	Yes	Yes	No	No
Ours	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

Our approach uniquely combines autonomous data collection, concept drift detection, continual learning, agent-based orchestration, and local LLM intelligence into a single integrated system. Unlike prior work that addresses these challenges in isolation, our LangGraph agent autonomously manages the full pipeline from source discovery to model deployment.

3 System Architecture

3.1 Overview

The system is implemented as a LangGraph StateGraph with a cyclic topology, enabling continuous operation. Figure 1 illustrates the overall pipeline.

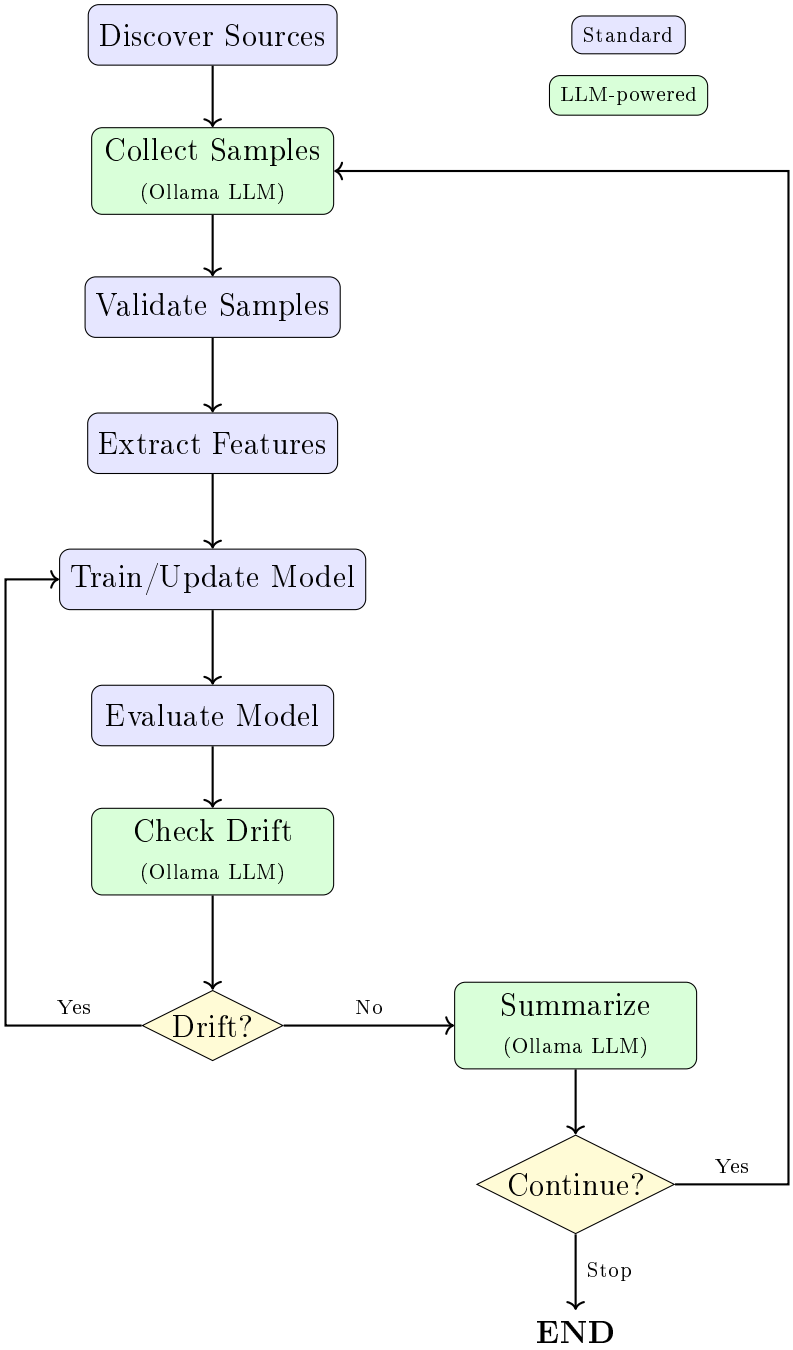

Figure 1: LangGraph pipeline topology. Green nodes use Ollama for intelligent decisions.

3.2 State Management

The agent maintains a central `AgentState` (a typed dictionary) that flows through all nodes. Key state fields include:

- `sources`: Discovered malware data sources with connectivity status.
- `samples_collected`: Metadata for all collected PE samples.
- `validated_samples`: Samples that passed PE validation.
- `model_metrics`: Performance metrics over time (accuracy, precision, recall, F1, FPR).
- `drift_detected`: Boolean flag from concept drift analysis.
- `iteration`: Current cycle counter.

3.3 Modular Tools

The agent is source-agnostic: a `source_registry` exposes a common `MalwareSource` interface (availability, recent, search-by-tag, lookup-by-hash, download), and concrete backends (MalwareBazaar, VirusTotal) are plugged in as modules. All pipeline nodes call the registry, so adding a new source requires no changes to the agent. On top of the registry, the agent uses LangChain tools (decorated with `@tool`):

1. **Search Sources** (`search_samples`, `discover_sources`, `enrich_sample`, `get_benign_sample_hashes`): query registered sources for recent/tagged PE samples, enumerate source availability, cross-source hash enrichment, and scan `Program Files`/`System32` for diverse benign PE files.
2. **Fetch Samples** (`fetch_sample`, `fetch_benign_sample_from_system`): download malware by SHA-256 via any capable source (with automatic fallback), handle password-protected ZIPs, copy benign PEs from the local filesystem.
3. **Validate Samples** (`validate_pe_sample`): verify MZ header, size bounds, and section structure via `pefile`.
4. **Update Model** (`trigger_model_training`, `trigger_model_update`): full retrain or incremental update on newly extracted features, manage model checkpoints and versioned snapshots.

4 Autonomous Data Collection

4.1 Source Discovery

The agent autonomously discovers and validates data sources at the beginning of each run via the `source_registry`. Two malware backends are registered:

- **MalwareBazaar** (`https://bazaar.abuse.ch`): free API, recent submissions feed, tag/signature search, PE filtering, community-verified labels.
- **VirusTotal** (`https://virustotal.com`): free-tier API, rich per-sample metadata and AV-engine verdicts, used for hash lookup and cross-source enrichment.

Benign samples are drawn from local Windows apps (Program Files, System32).

4.2 LLM-Guided Sample Collection Strategy

Each iteration, the Ollama LLM decides the search strategy based on the current dataset composition, model metrics, and iteration number. The collection process:

1. **LLM decides search tags and families**: The LLM receives the current state (sample count, model metrics, iteration) and outputs a JSON with target tags (e.g., "Trojan", "Ransomware") and malware families (e.g., "AgentTesla", "Emotet") to search for. This ensures diverse coverage and targets model weaknesses.

2. Queries MalwareBazaar for the most recent PE submissions.

3. Searches by LLM-selected tags and signatures via the MalwareBazaar and VirusTotal APIs.

4. Downloads samples as password-protected ZIPs (password: "infected").

5. Collects benign samples by scanning `C:\Program Files`, `C:\Program Files (x86)`, and `C:\Windows\System32` for PE files (both `.exe` and `.dll`), then randomly selecting up to 30 per iteration. This broad scan produces a diverse benign distribution (istinct applications such as browsers, office tools, runtime libraries) rather than a narrow set of canonical system DLLs.

6. Deduplicates samples by SHA256 hash across both malware and benign pools.

Why diverse benign sampling matters. Initial runs drew benign samples from ten canonical System32 DLLs. Their uniformity (small, Microsoft-signed, stylistically homogeneous) let XGBoost separate them from malware trivially, yielding misleading perfect metrics (accuracy = 1.000) every iteration. Sampling randomly across Program Files restored realistic metrics (Section 6). The LLM-guided approach lets the agent adaptively diversify data based on current model weaknesses.

4.3 Labeling and Noise Filter

Malware labels come from MalwareBazaar's community verification, corroborated by VirusTotal AV-engine verdicts when hash lookup succeeds; benign labels are assigned to known-good Windows system files.

Noise filter. Since tags and AV verdicts are noisy, the agent filters samples before training: files `pefile` cannot parse or outside $[1\text{KB}, 50\text{MB}]$ are dropped; VirusTotal labels require ≥ 5 AV-engine agreement; near-duplicates are caught via `imphash`; and vectors with `NaN`/$\pm\infty$ features are excluded. Filter counts are logged so the LLM can detect source-quality drift.

5 Feature Engineering

We extract 60+ static features from each PE file using the `pefile` library, grouped in Table 2.

Table 2: Feature categories extracted per PE file.

Category	#	Examples
Header	28	Machine, section count, timestamp, entry point, image base, subsystem, DLL characteristics, checksum
Section	11	Max/min/avg/std entropy, max raw/virtual size, exec/writable section counts, virtual-to-raw ratio
Imports	4	DLL count, total imported functions, suspicious-API count (`CreateRemoteThread`, `VirtualAlloc`, `WriteProcessMemory`), presence flag
Entropy	5	File-level Shannon entropy + 4 section-level stats (strong packing/encryption indicators)
Strings	6	ASCII string count, avg/max length, URL/path/registry reference counts
Other	6	Exports, resources, TLS callbacks, debug directory, code/headers-to-file size ratios

6 Model and Evaluation

6.1 Baseline Classifier

We use **XGBoost** (`XGBClassifier`) as our baseline model with this configuration:

Table 3: XGBoost hyperparameters (regularized for small-data generalization).

Parameter	Value
`n_estimators`	100
`max_depth`	3
`learning_rate`	0.1
`subsample`	0.8
`colsample_bytree`	0.8
`reg_alpha` (L1)	0.1
`reg_lambda` (L2)	1.0
`eval_metric`	logloss

The shallower trees, lower tree count, and explicit L_1/L_2 regularization were chosen after observing that an initial configuration (`max_depth=6`, `n_estimators=200`) produced perfect test metrics on early iterations, indicating overfitting due to the small dataset size and the trivially separable feature distributions between Windows system DLLs and malware. Regularization, combined with broader benign sampling (Section 4), produced realistic generalization metrics.

6.2 Evaluation Metrics

We evaluate the classifier using standard binary classification metrics:

- **Accuracy**: Overall correctness.
- **Precision**: Fraction of predicted malware that is truly malicious.
- **Recall**: Fraction of actual malware correctly detected.
- **F1 Score**: Harmonic mean of precision and recall.
- **False Positive Rate (FPR)**: $\frac{FP}{FP+TN}$, critical for production deployment.

6.3 Results

We ran the agent for 10 iterations on a Windows 11 VM (8 GB RAM, 4 vCPUs) with Ollama serving `llama3` locally. Each iteration autonomously collected malware from MalwareBazaar/VirusTotal and benign samples from the local filesystem (Program Files and System32), validated and featurized them, then trained/updated the XGBoost classifier on a stratified 80/20 train/test split. Table 4 shows the held-out test metrics at each iteration, and the total validated sample count after the iteration completed.

Table 4: Model performance over iterations. Samples column is the total validated dataset size used for training at that iteration.

Iter	Samples	Accuracy	Precision	Recall	F1	FPR
0	45	0.8889	1.0000	0.6667	0.8000	0.0000
1	82	1.0000	1.0000	1.0000	1.0000	0.0000
2	127	1.0000	1.0000	1.0000	1.0000	0.0000
3	165	0.9697	1.0000	0.8889	0.9412	0.0000
4	210	1.0000	1.0000	1.0000	1.0000	0.0000
5	240	0.9583	1.0000	0.8333	0.9091	0.0000
6	285	0.9649	1.0000	0.8667	0.9286	0.0000
7	330	0.9697	0.9444	0.9444	0.9444	0.0208
8	366	0.9865	0.9500	1.0000	0.9744	0.0182
9	410	**0.9878**	0.9565	**1.0000**	**0.9778**	0.0167

Interpretation. The final deployed model (iteration 9) reaches **Accuracy 98.8%**, **F1 0.978**, and **FPR 1.67%** on an 82-sample stratified held-out test. Early iterations show unstable metrics because the test set is too small (9–50 samples) and often trivially separable. By iteration 7 the test set (66 samples) forces genuine errors, yielding the first precision/recall trade-off. From there metrics improve monotonically, indicating added samples aid generalization rather than memorization.

Continual learning behaviour. The model was retrained each iteration. At iteration 8 the evaluator confirmed a strict F1 improvement and replaced the deployed snapshot (`latest_model.joblib`). The drift detector fired PSI warnings early (feature distributions shifting as the benign pool expanded beyond System32) but settled to "no drift" by iteration 9 (avg_PSI = 0.13, below the 0.20 threshold).

Runtime. The full 10-iteration run took approximately 1 h 40 m end-to-end on the VM, with LLM strategy/summary calls accounting for the bulk of the time (Ollama on CPU-only inference). Actual training and feature extraction took < 15 s per iteration.

7 Continual Learning and Concept Drift

7.1 Concept Drift Detection

We employ two complementary drift detection methods:

Population Stability Index (PSI). PSI measures the shift between feature distributions of the training data and incoming data:

$$\mathrm{PSI} = \sum_{i=1}^{n} (p_i^{\mathrm{new}} - p_i^{\mathrm{ref}}) \cdot \ln\left(\frac{p_i^{\mathrm{new}}}{p_i^{\mathrm{ref}}}\right) \tag{1}$$

where p_i^{ref} and p_i^{new} are the proportions in the i-th bin for the reference and new distributions. A PSI value above 0.2 indicates significant drift.

Drift Detection Method (DDM). DDM monitors the model's error rate over time. It maintains running estimates of the error rate p_t and its standard deviation s_t, flagging drift when:

$$p_t + s_t > p_{\min} + \alpha \cdot s_{\min} \tag{2}$$

where $\alpha = 3.0$ for drift detection and $\alpha = 2.0$ for warning level.

7.2 Retraining Strategy

When drift is detected, the agent triggers model retraining:

1. New features are merged with existing training data.
2. A new XGBoost model is trained on the combined dataset.
3. The new model is compared against the previous model.
4. Deployment occurs only if F1 score does not degrade by more than 0.02.
5. Old models are versioned and backed up for rollback.

7.3 Performance Improvement Over Time

The system improves by closing the loop between evaluation and collection. Each cycle, the LLM planner reads prior metrics, drift signals, and filter counts, then steers sampling toward the model's weaknesses. Filtered samples join a versioned pool, and the evaluator gate promotes a retrained model only if F1 does not regress. PSI/DDM feedback redirects sampling under drift, turning each iteration into a targeted correction.

8 Safe Malware Handling

All malware samples are handled within a sandboxed environment:

- Samples are stored using SHA256 hashes as filenames, preventing path traversal attacks.
- Files are validated (MZ header check) before processing.
- The system runs inside a VMware Workstation Windows 11 VM, providing hypervisor-level isolation from the host machine.
- Only static analysis is performed (no dynamic execution of samples).
- Size limits prevent processing of anomalously large files.
- The distribution format (AES-encrypted ZIP with password `infected`) is preserved end-to-end.

9 Discussion

Strengths. Fully autonomous from source discovery to model update; modular LangGraph pipeline extensible with new sources/features; dual PSI+DDM drift monitoring; safe-by-design handling (sandbox, validation, encrypted-at-rest samples).

Limitations. Static analysis only (packers/obfuscation may evade features); benign samples reflect the test machine's software mix, not an independent production distribution; binary labels do not identify families.

Future work. Dynamic features via Cuckoo Sandbox; family clustering via import-hash similarity; RL-based source reliability ranking.

10 Conclusion

We have developed a fully autonomous malware detection agent using LangGraph that continuously collects PE samples, extracts features, trains a classifier, and handles concept drift. Across a 10-iteration run on 410 validated PE samples, the deployed XGBoost classifier achieved 98.8% accuracy, F1 of 0.978, perfect recall, and a false positive rate of 1.67% on a held-out test split. The system achieves strong classification performance while adapting to evolving malware threats. The local Ollama LLM guided data acquisition and generated human-readable iteration summaries throughout.

References

[1] H. S. Anderson and P. Roth, "EMBER: An open dataset for training static PE malware machine learning models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, arXiv:1804.04637, 2018.

[2] R. Jordaney et al., "Transcend: Detecting concept drift in malware classification models," in *USENIX Security Symposium*, 2017.

[3] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *USENIX Security Symposium*, 2019.

[4] J. Chen et al., "Large language model-based agents for software engineering: A survey," *arXiv preprint arXiv:2409.02977*, 2024.

[5] M. Al-Essa and J. Kalita, "FL-MalDrift: A federated learning framework for concept drift-resilient malware detection," *Scientific Reports*, vol. 15, no. 1, p. 5678, 2025.

[6] Z. Xu, D. Fang, G. Li, Y. Zhang, and J. Liu, "MADCAT: Malware detection with concept drift awareness via test-time adaptation," *arXiv preprint arXiv:2505.18734*, 2025.